PROJECT ESTIMATION TECHNIQUES

COMP6100001 - SOFTWARE ENGINEERING
SESSION 12

UNIVERSITAS **BINA NUSANTARA**

SUBJECT MATTER EXPERT

Kenny Jingga, S.Kom., M.T.

SESSION 12 – PROJECT ESTIMATION TECHNIQUES

LEARNING OBJECTIVES

At the end of this session, students will be expected to be able to:

- Understand the Purpose and Importance of Project Estimation
- Describe Key Estimation Techniques and Their Applications
- Explore Software Sizing and Effort Estimation Models

SESSION 12 – PROJECT ESTIMATION TECHNIQUES

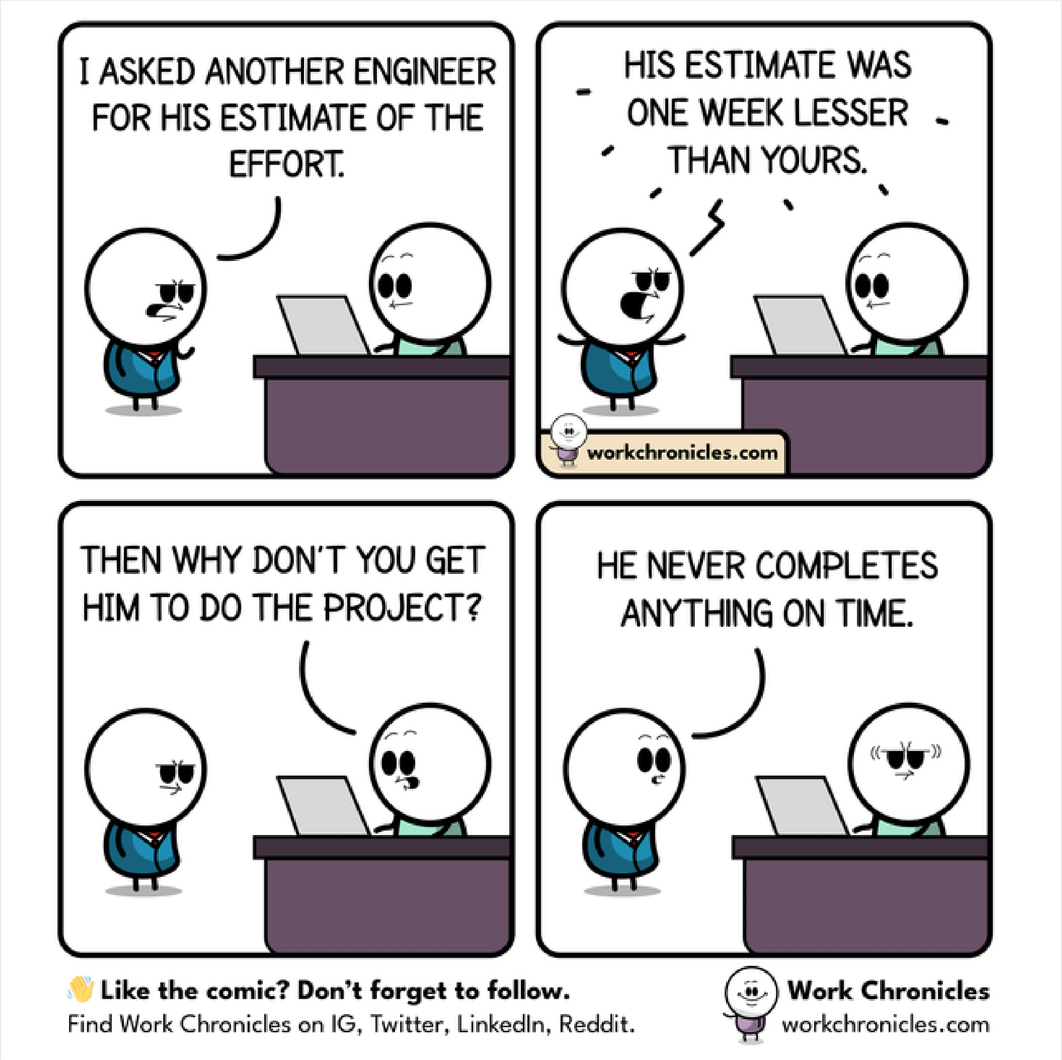
CONTENT OUTLINE

- Estimation Techniques
- Software Sizing Techniques
- Effort and Cost Estimation Models

ESTIMATION TECHNIQUES

EXPERT JUDGEMENT

- Definition: Relying on the experience and intuition of experts.
- Advantages: Quick and easy.
- Disadvantages: Subjective and may be biased.



ESTIMATION TECHNIQUES

ANALOGOUS ESTIMATION

- Definition: Using historical data from similar projects.
- Advantages: Useful when limited project details are available.
- Disadvantages: Accuracy depends on similarity between projects.

ESTIMATION TECHNIQUES

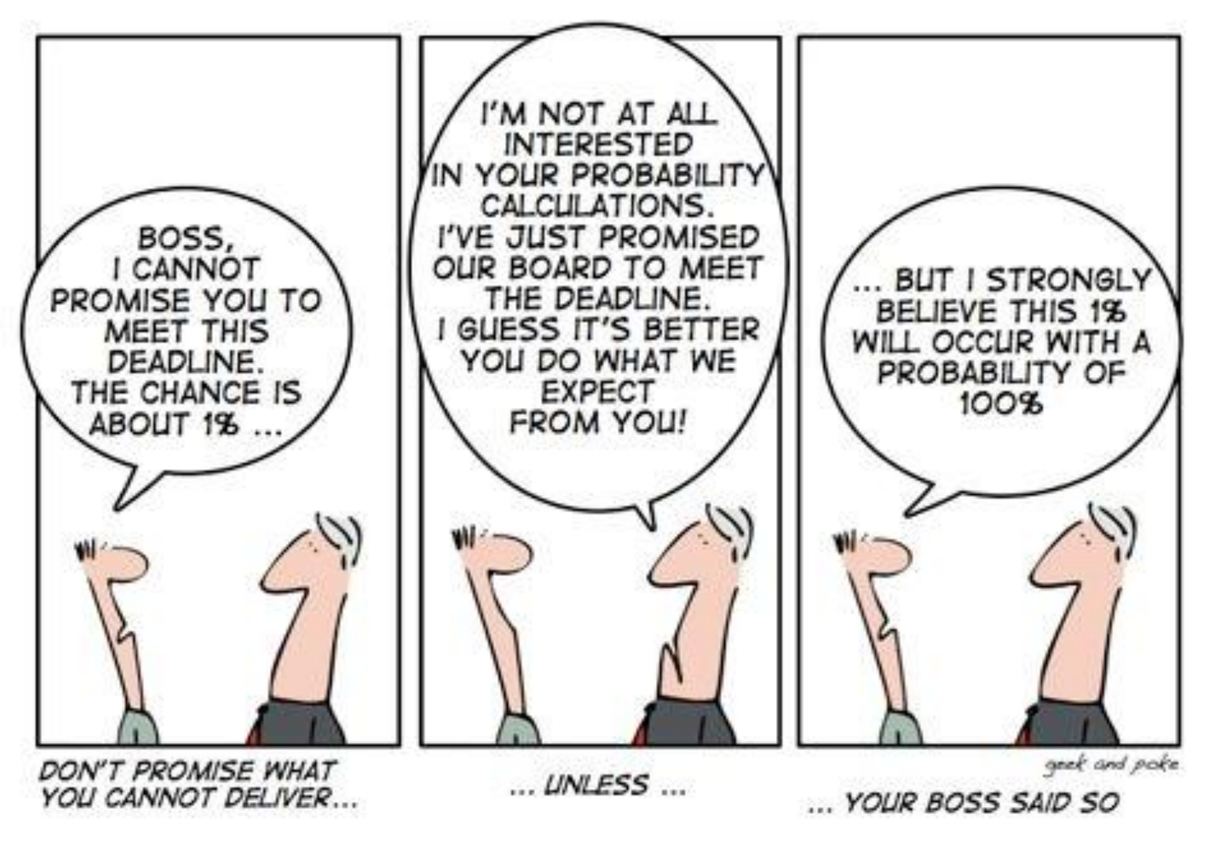
PARAMETRIC ESTIMATION

- Definition: Using mathematical models with project parameters.
- Example: Estimating effort based on lines of code (LOC).
- Advantages: More objective and scalable.
- Disadvantages: Requires accurate parameters and models.

ESTIMATION TECHNIQUES

THREE-POINT ESTIMATION (PERT)

- Definition: Uses optimistic (O), most likely (M), and pessimistic (P) estimates.
- Formula: Estimated Time = (O + 4M + P) / 6
- Advantages: Accounts for uncertainty.
- Disadvantages: More time-consuming.



ESTIMATION TECHNIQUES

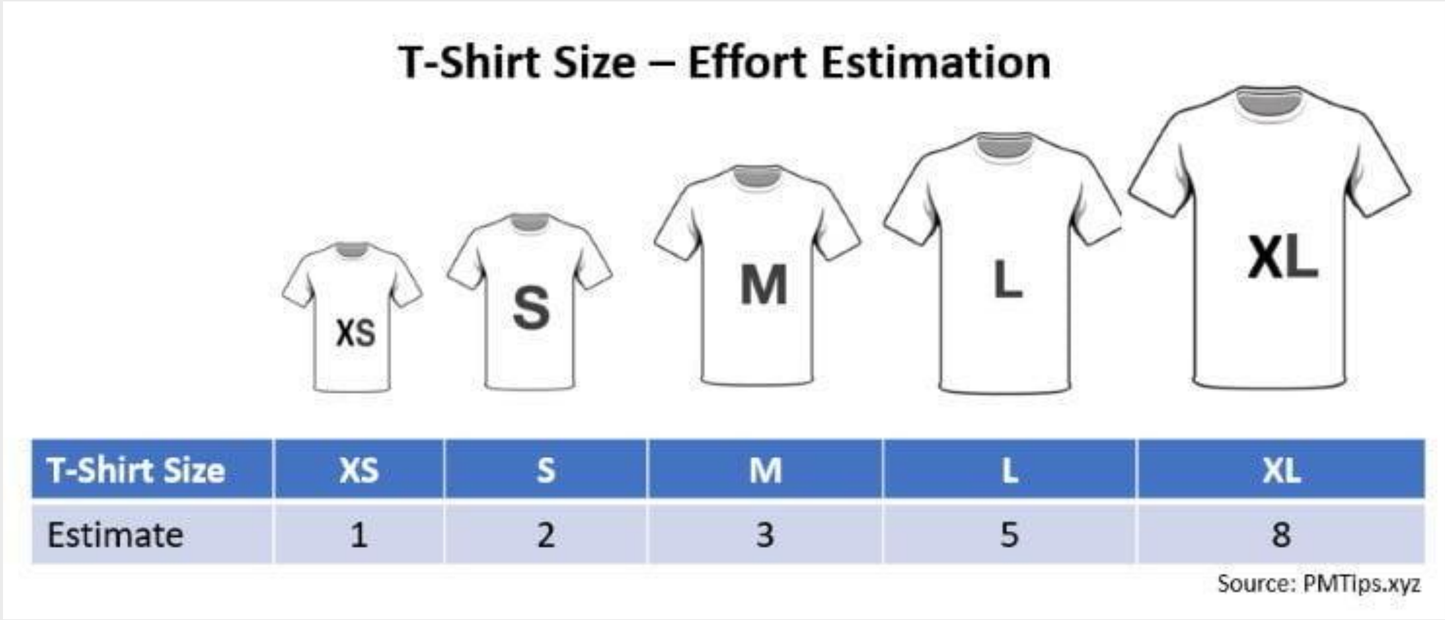
TIME ESTIMATION

THE AUTHOR OF THE WINDOWS FILE COPY DIALOG VISITS SOME FRIENDS.

ESTIMATION TECHNIQUES

IMPROVING TIME ESTIMATES

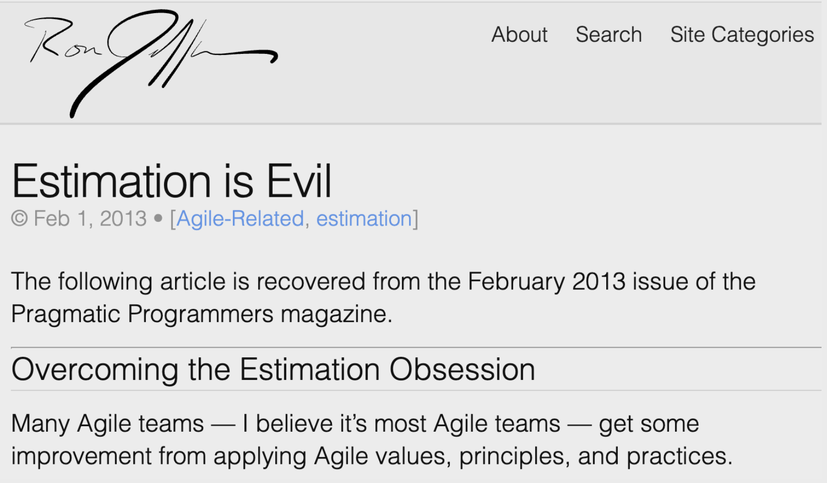
- Prevent conformity bias
- Do you have a comparable experience to base an estimate on?
- How much design do you need for each task?
- Break down the task into smaller tasks and estimate them



T-Shirt Size	XS	S	M	L	XL
Estimate	1	2	3	5	8

ESTIMATION TECHNIQUES

IS ESTIMATION EVIL?


About Search Site Categories

Estimation is Evil

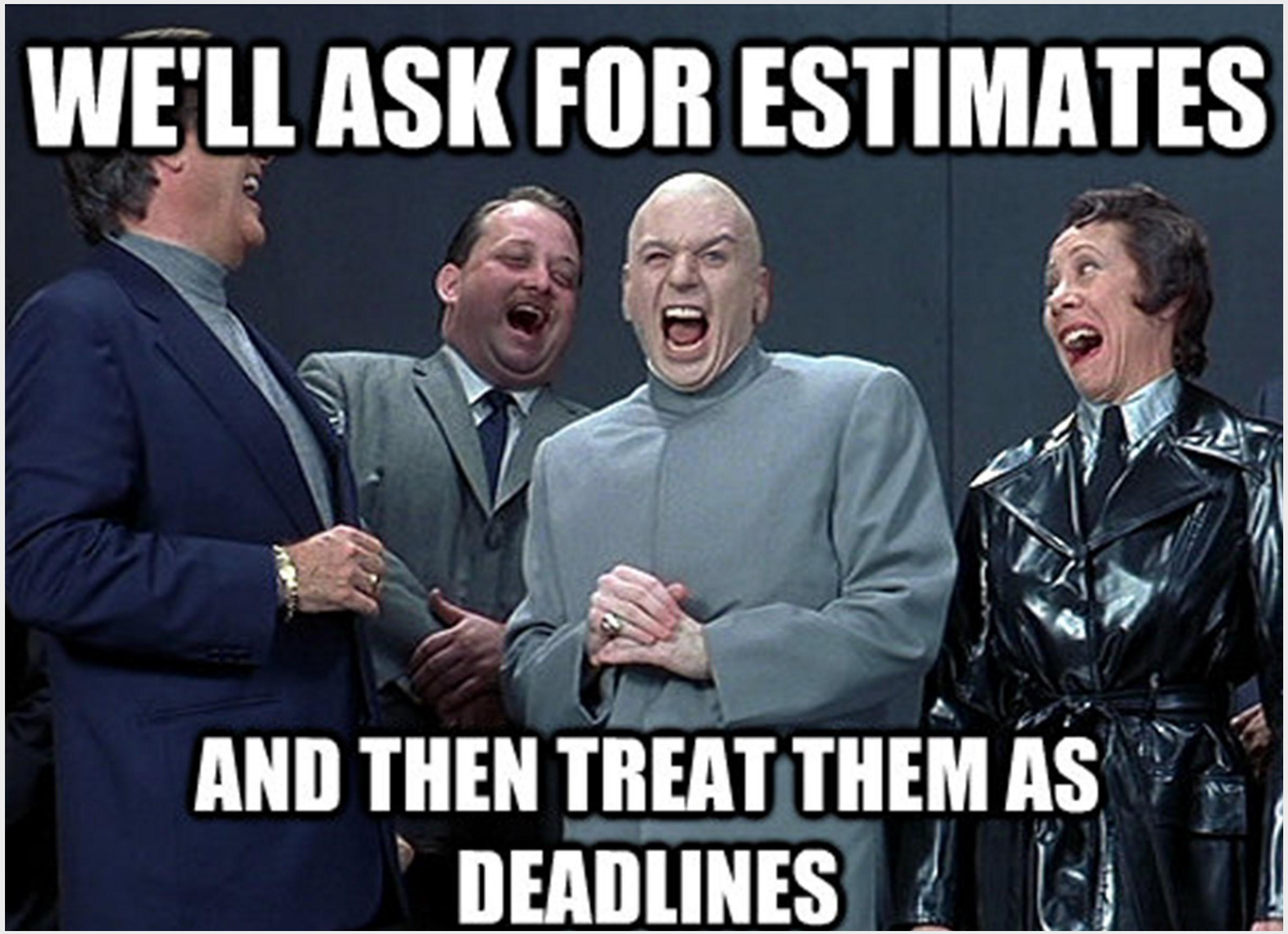
© Feb 1, 2013 • [Agile-Related, estimation]

The following article is recovered from the February 2013 issue of the Pragmatic Programmers magazine.

Overcoming the Estimation Obsession

Many Agile teams — I believe it's most Agile teams — get some improvement from applying Agile values, principles, and practices.

- https://ronjeffries.com/articles/021-01ff/estimation-is-evil/



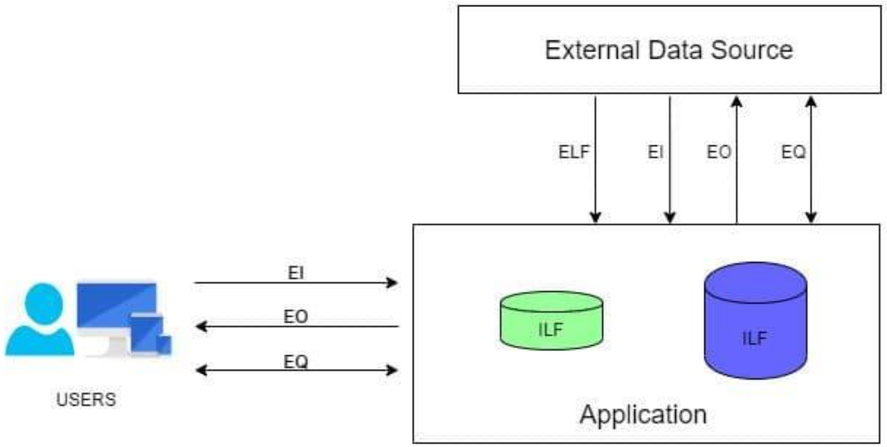
SOFTWARE SIZING TECHNIQUES

FUNCTION POINT ANALYSIS

- Measures the functionality provided to the user based on the logical design.

Components:

- External Inputs (EI)
- External Outputs (EO)
- External Inquiries (EQ)
- Internal Logical Files (ILF)
- External Interface Files (EIF)



Computing FPs

Measurement Parameter	Count		Weighing factor				
			Simple	Average	Complex		
1. Number of external inputs (EI)	—	*	3	4	6	=	—
2. Number of external Output (EO)	—	*	4	5	7	=	—
3. Number of external Inquiries (EQ)	—	*	3	4	6	=	—
4. Number of internal Files (ILF)	—	*	7	10	15	=	—
5. Number of external interfaces(EIF)	—	*	5	7	10	=	—
Count-total →							

SOFTWARE SIZING TECHNIQUES

FUNCTION POINT ANALYSIS

- Value Adjustment Factor (VAF) consists of 14 General System Characteristics (GSC)
- These GSCs represent characteristics of the application under consideration.
- Each is weighted on a scale from 0 (low) to 5 (high).
- When you sum up the values of these 14 GSCs you get a value named “Total Degree of Influence", or TDI.
- As you can see from the math the TDI can vary from 0 (when all GSCs are low) to 70 (when all GSCs are high).

14 GSCs:

- Data Communication
- Distributed data processing
- Performance
- Heavily used configuration
- Transaction rate
- Online data entry
- End user efficiency
- Online update
- Complex processing
- Reusability
- Installation ease
- Operational ease
- Multiple sites
- Facilitate change

$$\Sigma (F_i) = \text{Total Degree of Influence (TDI)}$$

$$VAF = [0.65 + 0.01 \times \Sigma (F_i)]$$

SOFTWARE SIZING TECHNIQUES

FUNCTION POINT ANALYSIS

Information domain value	Opt.	Likely	Pess.	Est. count	Weight	FP count
Number of external inputs	20	24	30	24	4	96 (24 x 4 = 96)
Number of external outputs	12	14	22	14	5	70 (14 x 5 = 70)
Number of external inquiries	16	20	28	20	5	100 (20 x 5 = 100)
Number of internal logical files	4	4	5	4	10	40 (4 x 10 = 40)
Number of external interface files	2	2	3	2	7	14 (2 x 7 = 14)
Count total						*320*

- The estimated number of FP is derived:

  Assume, $\Sigma (F_i) = 52$

  $FP_{estimated}$ = count-total x $[0.65 + 0.01 \times \Sigma (F_i)]$

  $FP_{estimated} = 374{,}4 = 375$

SOFTWARE SIZING TECHNIQUES

USE CASE POINTS

- Estimation technique based on use cases
- **Components**:
  - Unadjusted Use Case Weight (UUCW)
  - Unadjusted Actor Weight (UAW)
  - Technical Complexity Factors (TCF)
  - Environmental Factors (EF)
- **Calculation Steps**:
  - Calculate UUCW and UAW.
  - Compute Unadjusted Use Case Points (UUCP):
    - UUCP = UUCW + UAW
  - Adjust for TCF and EF.
  - Calculate Adjusted Use Case Points (UCP):
    - UCP = UUCP × TCF × EF

Use Case Weights

Use case complexity	Number of transactions	Weight
Simple	3 or fewer	5
Average	4 to 7	10
Complex	More than 7	15

Actor Complexity

Actor Type	Example	Weight
Simple	Another system through an API	1
Average	Another system through a protocal	2
	A person through a text-based user interface	
Complex	A person through through a graphical user interface	3

SOFTWARE SIZING TECHNIQUES

USE CASE POINTS

- Calculating Unadjusted Use Case Weight (UUCW)

Use case complexity	Weight	Number of use cases	Product
Simple	5	40	200
Average	10	21	210
Complex	15	10	150
Total			**560**

- Calculating Unadjusted Actor Weight (UAW)

Actor Type	Weight	Number of Actors	Product
Simple	1	8	8
Average	2	7	14
Complex	3	6	18
Total			**40**

- Calculating Unadjusted Use Case Points (UUCP): UUCW + UAW = 560+40 = 600

SOFTWARE SIZING TECHNIQUES

USE CASE POINTS: ADJUSTING FOR TECHNICAL COMPLEXITY

- The weight of each factor impacting technical complexity

Factor	Description	Weight
T1	Distributed system	2
T2	Performance objectives	2
T3	End-user efficiency	1
T4	Complex processing	1
T5	Reusable code	1
T6	Easy to install	0.5
T7	Easy to use	0.5
T8	Portable	2
T9	Easy to change	1
T10	Concurrent use	1
T11	Security	1
T12	Access for third parties	1
T13	Training needs	1

Factor	Weight	Assessment	Impact
Distributed system	2	3	6
Performance objectives	2	3	6
End-user efficiency	1	3	3
Complex processing	1	2	2
Reusable code	1	0	0
Easy to install	0.5	0	0
Easy to use	0.5	4	2
Portable	2	2	4
Easy to change	1	5	5
Concurrent use	1	5	5
Security	1	5	5
Access for third parties	1	3	3
Training needs	1	0	1
Total (TFactor)			**42**

- TCF = 0.6 + (0.01 * TFactor)
- TCF = 0.6 + (0.01 * 42) = 1.02

SOFTWARE SIZING TECHNIQUES

USE CASE POINTS: ADJUSTING FOR ENVIRONMENTAL COMPLEXITY

- Environmental factors and their weights

Factor	Description	Weight
E1	Familiar with the development process	1.5
E2	Application experience	0.5
E3	Object-oriented experience	1
E4	Lead analyst capability	0.5
E5	Motivation	1
E6	Stable requirements	2
E7	Part-time staff	-1
E8	Difficult programming language	-1

Factor	Weight	Assessment	Impact
Familiar with the development process	1.5	3	4.5
Application experience	0.5	4	2
Object-oriented experience	1	4	4
Lead analyst capability	0.5	4	2
Motivation	1	5	5
Stable requirements	2	1	2
Part-time staff	-1	0	0
Difficult programming language	-1	2	–2
Total (EFactor)			**17.5**

- EF = 1.4 + (-0.03 * EFactor)
- EF = 1.4 + (-0.03 * 17.5) = 1.4 + (-0.51) = 0.89

SOFTWARE SIZING TECHNIQUES

USE CASE POINTS: FINAL CALCULATION

Factor	Description	Weight
UUCW	Unadjusted Use Case Weight	560
UAW	Unadjusted Actor Weight	40
TCF	Technical Complexity Factor	1.02
EF	Environmental Factor	0.89

- UCP = UUCW * TCF * EF
- UCP = 600 * 1.02 * 0.89 = **545**

EFFORT AND COST ESTIMATION MODELS

LOC APPROACH

- compute LOC/FP using estimates of information domain values
- use historical data to build estimates for the project

Function	Estimated LOC
User interface and control facilities (UICF)	2300
Two-dimensional geometric analysis (2DGA)	5300
Three-dimensional geometric analysis (3DGA)	6800
Database management (DBM)	3350
Computer graphics display facilities (GCDF)	4950
Peripheral control function (PCF)	2100
Design analysis modules (DAM)	8400
Estimated lines of code	33200

- Average productivity for systems of this type = 620 LOC/pm.
- Burdened labor rate =$8000 per month, the cost per line of code is approximately $13.
- Based on the LOC estimate and the historical productivity data, the total estimated project cost is $431,000 and the estimated effort is 54 person-months.

EFFORT AND COST ESTIMATION MODELS

FP APPROACH

Information domain value	Opt.	Likely	Pess.	Est. count	Weight	FP count
Number of external inputs	20	24	30	24	4	96 (24 x 4 = 96)
Number of external outputs	12	14	22	14	5	70 (14 x 5 = 70)
Number of external inquiries	16	20	28	20	5	100 (20 x 5 = 100)
Number of internal logical files	4	4	5	4	10	40 (4 x 10 = 40)
Number of external interface files	2	2	3	2	7	14 (2 x 7 = 14)
Count total						*320*

- The estimated number of FP is derived:

  $FP_{estimated} = 375$
- organizational average productivity = 6.5 FP/pm.
- burdened labor rate = $8000 per month, approximately $1230/FP.
- Based on the FP estimate and the historical productivity data, total estimated project cost is $461,000 and estimated effort is 58 person-months.

EFFORT AND COST ESTIMATION MODELS

COCOMO II MODEL

- **Constructive Cost Model**, an algorithmic software estimation model.
- COCOMO II is an advanced version of the original COCOMO model developed by Dr. Barry Boehm in 1981. It was introduced in the mid-1990s to address the changing landscape of software development, including:
  - Emergence of new software development methodologies.
  - Increased use of Commercial Off-The-Shelf (COTS) software.
  - Importance of software reuse.
  - Development of software in a variety of domains and environments.
- **Purpose of COCOMO II**:
  - Provide a framework for estimating software development effort, cost, and schedule.
  - Adapt to various project sizes, complexities, and development approaches.
- **Types**:
  - **Application Composition Model**: For early prototyping stages.
  - **Early Design Model**: Uses function points and a set of scale drivers.
  - **Post-Architecture Model**: Most detailed, uses size estimates in LOC.

EFFORT AND COST ESTIMATION MODELS

COCOMO II MODEL: APPLICATION COMPOSITION MODEL

- **Usage**: Suitable for projects during the early stages where software is built using rapid application development (RAD) tools or is component-based.

- **Approach**: Utilizes Object Points (OP) as the primary size metric.

- **Object Points Categories**:
  - Screens: User interface elements.
  - Reports: Outputs generated by the system.
  - Components: Reusable pieces of code or modules.

- **Effort Estimation Formula**:

$$Effort\ (Person - Months) = \frac{Object\ Points\ \times\ (100\%\ -\ Reuse\%)}{Productivity\ Rate}$$

  - **Productivity Rate**: Based on developer experience and tool support.

EFFORT AND COST ESTIMATION MODELS

COCOMO II MODEL: EARLY DESIGN MODEL

- **Usage**: Applied during the early stages when only high-level design information is available.
- **Approach**: Uses **Function Points (FP)** or **Unadjusted Function Points (UFP)** as the size metric.
- **Key Components**:
  - **Scale Factors (SF)**: Reflect the exponential impact of project characteristics.
  - **Effort Multipliers (EM)**: Adjust effort based on cost drivers.
- **Effort Estimation Formula**:

$$Effort = EAF \times Size^{E}$$

  - **EAF (Effort Adjustment Factor)**: Product of Effort Multipliers.
  - **E (Exponent)**: Calculated using Scale Factors.

EFFORT AND COST ESTIMATION MODELS

COCOMO II MODEL: POST-ARCHITECTURE MODEL

- **Usage**: Suitable after the project's overall architecture has been designed.
- **Approach**: Provides the most detailed estimation, incorporating a wide range of factors.
- **Size Metric**: Typically uses **Source Lines of Code (SLOC)** or **Function Points**.

- **Key Components**:
  - **17 Effort Multipliers (EM):** Cover product, platform, personnel, and project attributes.
  - **5 Scale Factors (SF):** Capture the economy or diseconomy of scale.

- **Effort Estimation Formula**:

$$Effort(Person - Months) = EAF \times Size^{E}$$

  - **EAF (Effort Adjustment Factor)**: Product of the selected Effort Multipliers.
  - **E (Exponent)**: Exponent derived from Scale Factors:

$$E = B + 0.01 \times \sum_{i=1}^{5} SF_i$$

  - **B**: Baseline exponent, typically 0.91

EFFORT AND COST ESTIMATION MODELS

DETAILED COMPONENTS OF COCOMO II: SCALE FACTORS (SF)

- Scale Factors determine the economies or diseconomies of scale in the project. The five Scale Factors are:
  - **PREC (Precedentedness)**: Reflects previous experience with similar projects.
  - **FLEX (Development Flexibility)**: Degree of flexibility in the development process.
  - **RESL (Architecture/Risk Resolution)**: Level of risk management and architecture analysis.
  - **TEAM (Team Cohesion)**: How well the team works together.
  - **PMAT (Process Maturity)**: Maturity of the organization's software process.
- **Rating Levels**: Each Scale Factor is rated on a scale from Very Low to Extra High, with associated numeric values.

EFFORT AND COST ESTIMATION MODELS

DETAILED COMPONENTS OF COCOMO II: SCALE FACTORS (SF)

- Example:

Scale Factor	Very Low	Low	Nominal	High	Very High	Extra High
PREC	6.20	4.96	3.72	2.48	1.24	0.00
FLEX	5.07	4.05	3.04	2.03	1.01	0.00
RESL	7.07	5.65	4.24	2.83	1.41	0.00
TEAM	5.48	4.38	3.29	2.19	1.10	0.00
PMAT	7.80	6.24	4.68	3.12	1.56	0.00

- The sum of these values is used to calculate the exponent E.

EFFORT AND COST ESTIMATION MODELS

DETAILED COMPONENTS OF COCOMO II: EFFORT MULTIPLIERS (EM)

- Effort Multipliers adjust the estimated effort based on specific project attributes. There are 17 EMs divided into four categories:
- **Product Factors**:
  - **RELY** (Required Software Reliability)
  - **DATA** (Database Size)
  - **CPLX** (Product Complexity)
  - **RUSE** (Required Reusability)
  - **DOCU** (Documentation Match to Life-Cycle Needs)
- **Platform Factors**:
  - **TIME** (Execution Time Constraint)
  - **STOR** (Main Storage Constraint)
  - **PVOL** (Platform Volatility)
- **Personnel Factors**:
  - **ACAP** (Analyst Capability)
  - **PCAP** (Programmer Capability)
  - **PCON** (Personnel Continuity)
  - **APEX** (Applications Experience)
  - **PLEX** (Platform Experience)
  - **LTEX** (Language and Tool Experience)
- **Project Factors**:
  - **TOOL** (Use of Software Tools)
  - **SITE** (Multisite Development)
  - **SCED** (Required Development Schedule)

Drivers	Very low	Low	Nominal	High	Very high	Extra high
RELY	0.82	0.92	1.00	1.10	1.26	–
DATA	–	0.90	1.00	1.14	1.28	–
CPLX	0.73	0.87	1.00	1.17	1.34	1.74
RUSE	–	0.95	1.00	1.07	1.15	1.24
DOCU	0.81	0.91	1.00	1.11	1.23	–
TIME	–	–	1.00	1.11	1.29	1.63
STOR	–	–	1.00	1.05	1.17	1.46
PVOL	–	0.87	1.00	1.15	1.30	–
ACAP	1.42	1.19	1.00	1.85	0.71	–
PCAP	1.34	1.15	1.00	0.88	0.76	–
PCON	1.29	1.12	1.00	0.88	0.81	–
APLEX	1.22	1.10	1.00	0.90	0.81	–
APLEX	1.19	1.09	1.00	0.91	0.85	–
LTEX	1.20	1.09	1.00	0.91	0.84	–
TOOL	1.17	1.09	1.00	0.90	0.78	–
SITE	1.22	1.09	1.00	0.93	0.86	0.80
SCED	1.43	1.14	1.00	1.00	1.00	–

- **Rating Levels**: Similar to Scale Factors, each EM is rated from Very Low to Extra High, with corresponding numeric multipliers.
- The EMs are multiplied together to obtain the EAF.

STEPS FOR USING THE POST-ARCHITECTURE MODEL

STEP 1: ESTIMATE SIZE

- Determine the size of the software to be developed.
- **Size Metrics**: Use SLOC or Function Points.
- Considerations:
  - Exclude reused code (unless significant modifications are required).
  - Include new code to be developed.

STEPS FOR USING THE POST-ARCHITECTURE MODEL

STEP 2: RATE SCALE FACTORS

- Evaluate each of the five Scale Factors based on project characteristics.
- Use the provided rating levels and associated values.
- **Calculate E (Exponent)**:

$$E = B + 0.01 \times \sum_{i=1}^{5} SF_i$$

- **B**: Baseline constant (commonly 0.91 in COCOMO II).

STEPS FOR USING THE POST-ARCHITECTURE MODEL

STEP 3: RATE EFFORT MULTIPLIERS

- Assess each of the 17 Effort Multipliers.
- Assign the appropriate rating based on the project's context.
- Calculate EAF:

$$EAF = \prod_{i=1}^{17} EM_i$$

STEPS FOR USING THE POST-ARCHITECTURE MODEL

STEP 4: CALCULATE EFFORT

- **Effort Estimation Formula**:

$$Effort(Person - Months) = EAF \times Size^{E}$$

- Substitute the values of EAF, Size, and E into the formula

STEPS FOR USING THE POST-ARCHITECTURE MODEL

STEP 5: ESTIMATE SCHEDULE AND STAFFING

- **Schedule Estimation Formula**:

$$Schedule(Months) = C \times Effort^F$$

- C: Schedule coefficient (commonly 3.67).
- F: Schedule scaling factor (commonly 0.28 + 0.2 × (E - B)).
- **Average Staffing**:

$$Staffing = \frac{Effort}{Schedule}$$

REFERENCES

- Titus Winters, Tom Manshreck, and Hyrum Wright. (2020). ***Software Engineering at Google: Lesson Learned from Programming Over Time***. 1st. O'Reilly Media. California. ISBN: 978-1492082798. Page 69-122.
- Roger Pressman and Bruce Maxim. (2020). ***Software Engineering: A Practitioner's Approach***. 9th. McGraw Hill. New York. ISBN: 978-1259872976. Chapter 24.
- Ian Sommerville. (2016). ***Software Engineering***. 10th. Pearson. London. ISBN: 978-1-292-09613-1. Chapter 22.